

Control

Programming and Music
<http://www.bjarni-gunnarsson.net>

Mapping

Mapping

The process of transferring control values to musical parameters. Is often discussed in terms of four different categories:

- * *One to One*

- * *One to Many*

- * *Many to One*

- * *Many to Many*

Mapping

There exists endless possibilities of connecting information from external processes or controls to musical parameters. It is very important to consider the **mapping** carefully and in many cases the mapping itself is where the magic happens.

In the field of algorithmic composition mapping is less discussed than properties or behavior of generative algorithms.

It is sometimes assumed mapping is personal or full of irrelevant details.

Mapping

Mapping is still what connects external controls to musical parameters and can be considered to be part of ***compositional choices*** a composer has to make.

When mapping choices are made part of a system the system can be thought of as a ***compositional system*** or ***composed instrument***.

Mapping

Compositional choices are focused on **planning** but also on setting **limits** and creating **boundaries**. This overlaps somehow with instrument design and it is an interesting to discover where one considers his mapping to exist.

Connecting sound processes to compositional choices creates an **abstraction** that is essential to mapping procedures. What is mapped and how the behavior of the mapping is implemented has an important effect on choices made both for sound generation and how ones compose for a given system.

Considerations

- * Does a mapping make sense intuitively ?
- * Does the mapping curve respond well to its domain ?
- * Is the control range expressive enough ?
- * Does the control range contain the right space for change ?
- * Can the mapping effect be easily remembered ?
- * Does a mapping relate well to other mappings in the system ?
- * How isolated or dependent is a mapping to other controls ?

Scaling and Offsetting

Scaling a quantity means to *change its size*, or magnitude, by multiplication. Scaling an object visually would mean to make it appear bigger or smaller without altering its origin.

Offsetting a quantity means to add a value to it. Offsetting affects the origin of something without affecting the size.

Scale and Offset are important mapping procedures and many UGens in SuperCollider take these as standard arguments.

Curves and Envelopes

Transferring one range of values to another can either be done **linearly** or by using a **transfer function**.

The envelope is a very simple yet powerful data structure in sound synthesis and can be used also for parameter mapping. Using envelopes one can define **custom transfer functions** from one domain to the other.

A **normalized envelope** is an envelope where its values have a sum of 1. Using a normalized envelope is recommended since the range can easily be changed independently of the envelope values.

Processing

In addition to mapping curves or procedures, data can also be both ***pre-processed*** and/or ***post-processed***.

Pre-processing tasks could look at incoming data and ***remove*** or ***replace*** certain items. For big data sets it could also ***select*** or ***reject*** items based on a given criteria.

Post-processing tasks would involve treatment of data such as making values ***last*** or ***lag*** for some duration. ***Delay*** on control signals could also be introduced as well as ***interpolation*** between new and old values.

(from ***Mapping and visualization with SuperCollider***

<http://marinoskoutsoomichalis.com/mapping-and-visualization/>)

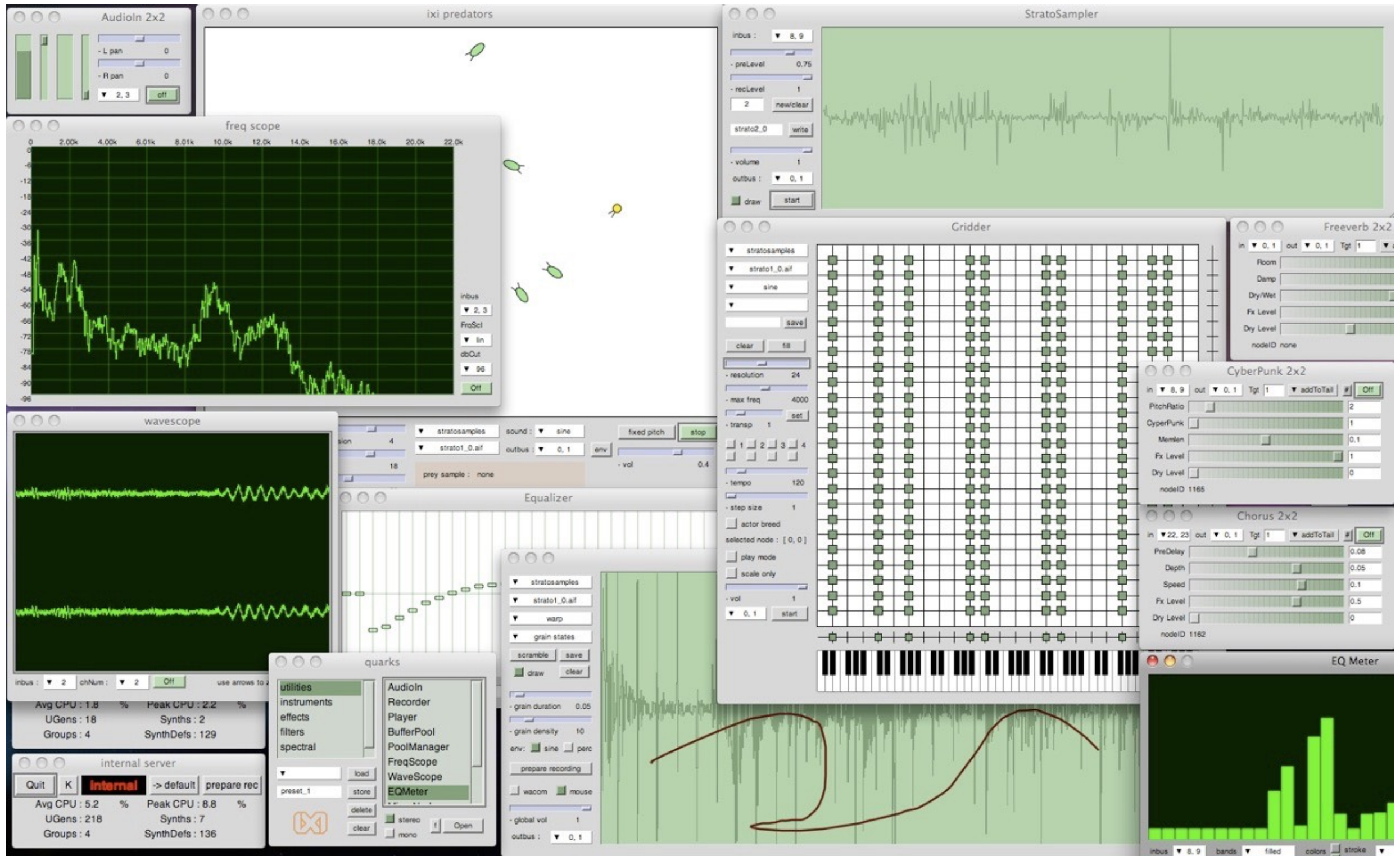
MIDI

Information about MIDI support in SuperCollider can be found in the help file ***UsingMIDI*** and the help file ***MIDI***.

Using ***MIDIFunc***, functions can be defined to respond to received MIDI messages. Several convenience methods are available to deal with specific MIDI messages such as ***noteOn***, ***cc*** etc.

The ***MIDIClient*** class handles MIDI connection and communication with the operating system. It has to be initialized and once functions have been registered they have to be freed if they should no longer be called.

SuperCollider



Why use a GUI ?

More visually intuitive.

Learning how to use a system is quicker.

Access to parameters is quick.

Multitasking is more straightforward.

Lets one focus on available controls instead of code.

Visible controls and screen space force abstraction.

Code potentially changes more frequently than a GUI.

SC GUI

Originally, SuperCollider was a ***MacOS application***. GUI elements had class names such as SCWindow and SCSlider. These elements looked like ***native*** MacOS GUI elements.

A focus on making ***cross-platform*** GUIs led to the use of more generic class names such as Window and Slider. Methods could be used to choose the type of GUI.

Currently (SC 3.6.3 with new IDE), SuperCollider has migrated to using the ***QT cross-platform framework*** and the use of Macintosh native elements (such as SCWindow) is no longer possible.

Various other features related to creating GUIs are also in transition.

SC GUI

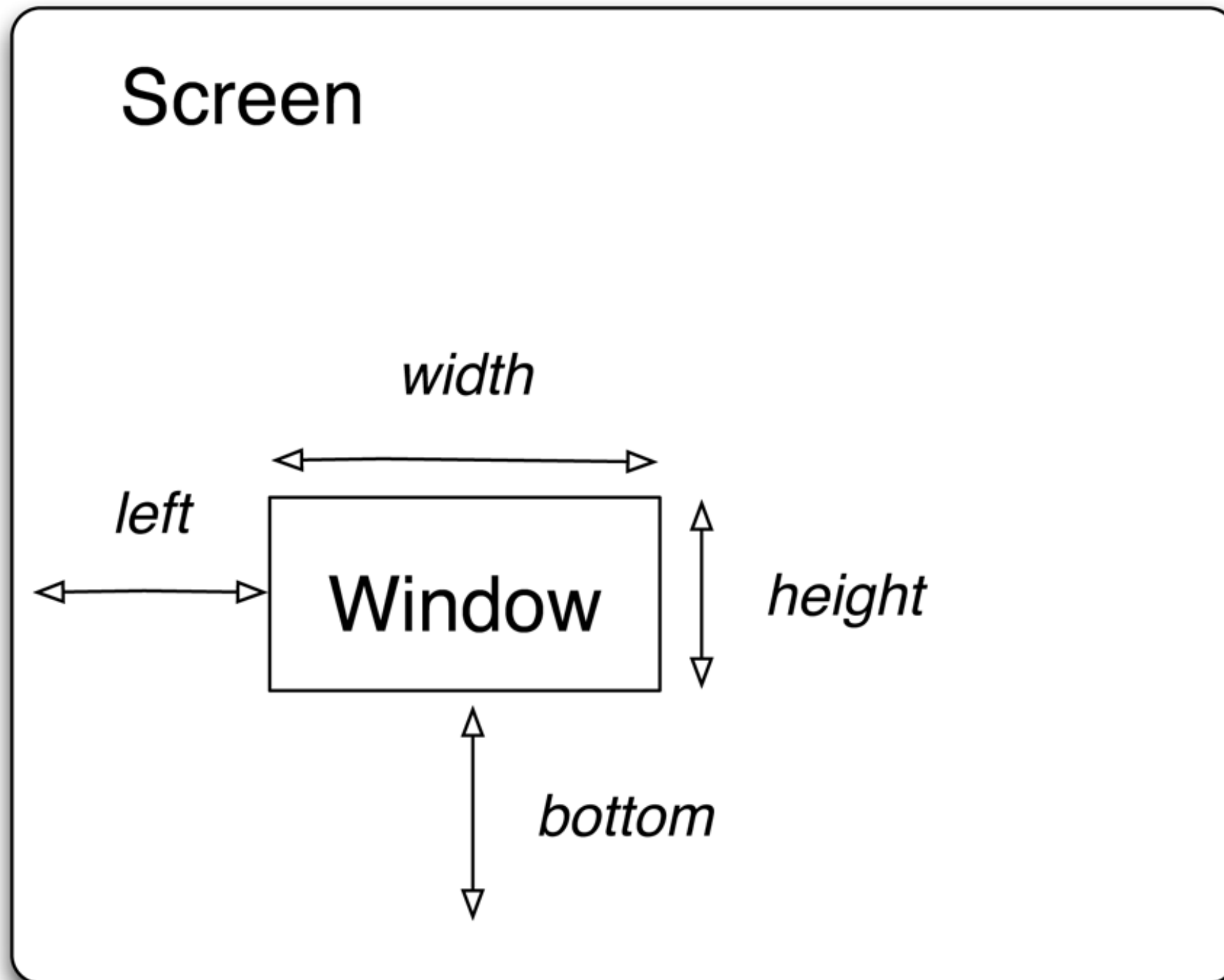
Some information about creating a GUI can be found in the Document Browser, under the category: GUI. This documentation may not be up to date.

In most cases, a user will need to:

- * Create a ***window***
- * Insert ***items*** such as sliders, buttons, etc.
- * Specify ***what*** each of these items does

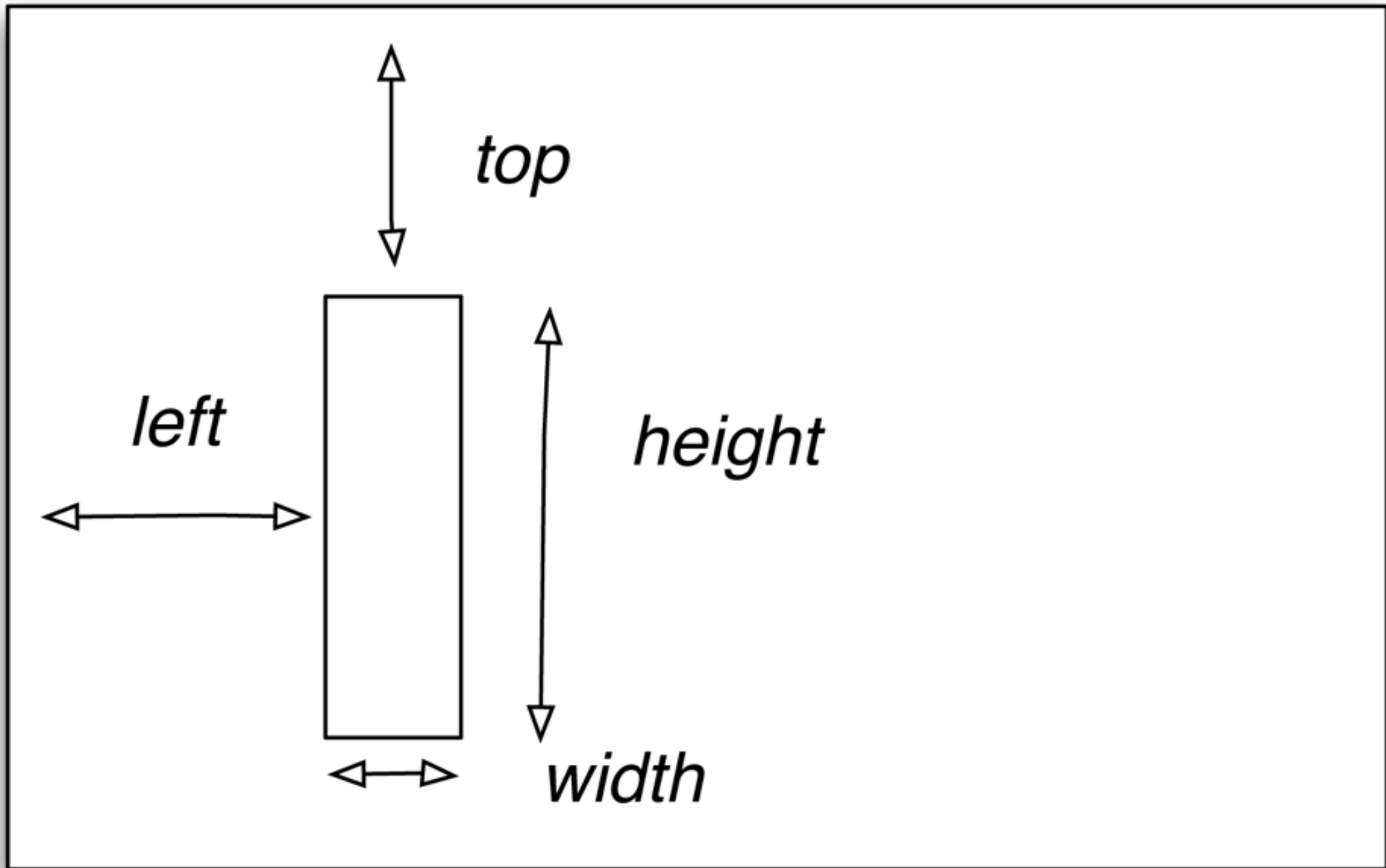
There is also an option to automatically 'decorate' a window which may make things easier.

Position of window in a screen



Rect(left, bottom, width, height)

Position of object in a window



Rect(left, top, width, height)

Mapping with Control Spec

Map



Unmap



```

Pstep3add : Pattern {
  <pattern1, <pattern2, <pattern3;
  *new { pattern1, pattern2, pattern3;
    ^ .newCopyArgs(pattern1, pattern2, pattern3)
  }
  storeArgs { ^[pattern1,pattern2,pattern3] }

  embedInStream {
    inval;
    stream1, stream2, stream3, val1, val2, val3;

    stream1 = pattern1.asStream;
    while {
      (val1 = stream1.next(inval)).notNil;
    }{
      stream2 = pattern2.asStream;
      while {
        (val2 = stream2.next(inval)).notNil;
      }{
        stream3 = pattern3.asStream;
        while {
          (val3 = stream3.next(inval)).notNil;
        }{
          inval = yield( val1 + val2 + val3 );
        }
      };
    };
  };
  ^inval;
}

```

Exercises

```

PstepNfunc : Pattern {
  <function, <patterns;
  *new { function, patterns;
    ^ .newCopyArgs(function ? { |x| x }, patterns)
  }
  storeArgs { ^[function,patterns] }
  embedInStream {
    inval;
    val;
    size = patterns.size;
    max = size - 1;
    streams = Array.newClear(size);
    vals = Array.newClear(size);
    // this variable is needed because of recursion
    f = {
      inval, level=0;
      val;
      streams[level] = patterns[level].asStream;
      while{
        vals[level] = val = streams[level].next(inval);
        val.notNil;
      }
    }
  }
}

```

Exercises - GUI

1. Implement a trigger-based GUI where a touch of a button will trigger a sound to play. The interface could include additional settings such as the attack and release of each sound.
2. Create a GUI for simple sample playback including the loading of a sample.
3. Create a GUI that has a drag-and-drop interface for choosing SynthDefs to play.
4. Create a GUI that allows its user to generate envelopes that she can later edit visually.

Exercises - MIDI & Mapping

- 1. Create a one to many mapping for a synthesis process where the transfer function is different from the standard linear one.*
- 2. Create a mapping where the results are not audible until at least several seconds after the input control has been stopped.*
- 3. Create a mapping where an incoming signal is used to control a pattern and a synth at the same time.*
- 4. Create a mapping using two trajectories for one input signal where the result is a random number between the two (a tendency mask).*

